Consola para Administración de Máquinas Virtuales GNU/Linux

Jorge Acosta Andrey Tombe

Universidad del Quindío
Facultad de Ingeniería
Ingeniería de Sistemas y Computación
Computación en la Nube
Octubre del 2026

***Resumen*—(Redactar al final del informe.)**

***Index Terms*—palabra clave, palabra clave, palabra clave**

I. Introducción

La administración de sistemas operativos GNU/Linux es una actividad fundamental en el área de computación en la nube. Tareas como la instalación de paquetes, la configuración del servidor SSH, la gestión de servicios y el balanceo de carga se realizan, en muchos entornos, de manera manual y a través de comandos introducidos directamente en una terminal. Este enfoque, si bien es válido, resulta propenso a errores de digitación, difícil de estandarizar y poco accesible para usuarios que no dominan la línea de comandos.

A lo largo del curso de Computación en la Nube se han ejecutado estas operaciones de forma tradicional. En el presente trabajo se propone convertir dichas tareas en operaciones que pueden solicitarse desde una aplicación web desarrollada en el lenguaje Go. La aplicación funciona como una consola de automatización para una máquina virtual que ejecuta GNU/Linux Debian 13, integrando las operaciones del sistema en una interfaz de usuario que permite validar las entradas, ejecutar las acciones de manera no interactiva y reportar el resultado de cada proceso, incluyendo mensajes de error comprensibles y la información técnica necesaria para el diagnóstico.

II. Marco Teórico

II-A. Máquinas virtuales y virtualización

La virtualización permite ejecutar varios sistemas operativos aislados sobre un mismo equipo físico. Un hipervisor intermedia el acceso a la CPU, a la memoria y a las interfaces de red de modo que cada sistema operativo invitado cree estar frente a una máquina propia. En este trabajo se utiliza Oracle VM VirtualBox, un hipervisor de tipo 2, es decir, alojado sobre el sistema operativo anfitrión ya instalado en el equipo, y multiplataforma, con soporte para GNU/Linux y Windows [1].

VirtualBox se administra tanto desde su interfaz gráfica como desde la utilidad de línea de comandos `VBoxManage`, incluida en toda instalación y con el mismo comportamiento en todos los sistemas operativos soportados [1]. Esta última característica es la que aprovecha la consola: cualquier operación que la aplicación deba realizar sobre una máquina virtual —consultar su estado, obtener su dirección de red, arrancarla o apagarla— puede expresarse como una llamada a `VBoxManage`, sin depender de la ventana gráfica ni de ninguna herramienta adicional.

Las Guest Additions son controladores y servicios que se instalan dentro de la máquina virtual y mejoran su integración con el anfitrión. Entre otras funciones, publican propiedades del sistema invitado, como la dirección IP de la interfaz de red, que el anfitrión puede leer con `VBoxManage guestproperty get` a través de la propiedad `/VirtualBox/GuestInfo/Net/0/V4/IP` [2]. Este mecanismo resuelve un problema práctico: la consola necesita una dirección para conectar por SSH, pero mientras SSH no esté instalado y activo no existe ningún canal de comunicación con la máquina. Las Guest Additions cubren esa primera conexión y, una vez instalado SSH, dan paso al canal con el que se trabaja el resto del tiempo.

II-B. Distribución Debian y gestión de paquetes con APT

Todas las máquinas virtuales del escenario ejecutan GNU/Linux Debian 13.7.0, de la rama estable *trixie* [3], distribución conocida por su estabilidad y por su gestor de paquetes. Debian sigue la jerarquía estándar de directorios de GNU/Linux: la configuración vive en `/etc`, los programas en `/usr/bin` y `/usr/sbin` y los datos variables en `/var`. Varios de esos archivos son el destino directo de los módulos de la consola: `/etc/issue` guarda el mensaje que se muestra antes del inicio de sesión y admite comandos que expanden el nombre del host, la fecha, la hora o la dirección IP; `/etc/hostname` y `/etc/hosts` guardan el nombre de la máquina y su resolución local de nombres; y `/etc/ssh/sshd_config`

junto con el directorio `~/.ssh/` concentran la configuración y las llaves del servicio SSH.

APT (*Advanced Package Tool*) es la herramienta de nivel superior con la que Debian instala, actualiza y elimina paquetes `.deb` [4]. Los paquetes se descargan de los repositorios declarados en `/etc/apt/sources.list` y en los archivos de `/etc/apt/sources.list.d/`; `apt update` sincroniza el índice local con el repositorio, `apt upgrade` actualiza los paquetes instalados, `apt install` instala uno o varios paquetes resolviendo sus dependencias y `apt remove` los desinstala (`apt purge` elimina además sus archivos de configuración).

Un detalle relevante para este proyecto es que APT, en su uso normal, consulta al usuario antes de instalar o desinstalar. Como el enunciado exige que la consola opere de forma no interactiva, la herramienta debe invocarse con la opción `-y` o con la variable `DEBIAN_FRONTEND` establecida a `noninteractive`; de ese modo confirma las acciones por sí misma y devuelve en la salida estándar y en su código de salida el resultado que la aplicación traduce en el mensaje que ve el usuario. Estas órdenes sostienen la precondición (a), una máquina actualizada con `apt update && apt upgrade`, y el módulo de instalación y eliminación de aplicaciones.

II-C. Protocolo y servicio SSH

SSH (*Secure Shell*) proporciona canales seguros sobre redes no confiables, con autenticación cifrada, confidencialidad e integridad de las comunicaciones [5]. En sistemas GNU/Linux su implementación de referencia es OpenSSH, que ofrece el demonio `sshd`, encargado de atender las conexiones, y el cliente homónimo, encargado de abrir sesiones y ejecutar órdenes en la máquina remota. Instalar y habilitar `sshd` en cada máquina virtual es lo que permite que la consola se conecte a ellas por red.

Además de la autenticación por contraseña, SSH admite la de par de llaves asimétricas: el cliente conserva la llave privada, el servidor guarda la pública en `~/.ssh/authorized_keys` y comprueba la firma sin que viaje ningún secreto por la red. La contraseña puede deshabilitarse por completo [5], de modo que el acceso queda protegido únicamente por la posesión de la llave. Éste es el mecanismo que la consola configura cuando genera las llaves de root y las instala en la máquina virtual.

El cliente SSH también sirve como interfaz de automatización: ejecuta órdenes de forma no interactiva con la sintaxis `ssh usuario@host orden` y devuelve el código de salida del programa remoto, y transfiere archivos con `scp`. Gracias a ello la aplicación puede administrar cualquier máquina sin intervención de un operador, y todo lo que hace dentro de la máquina —ya sea instalando un paquete, escribiendo un archivo de configuración o reiniciando un servicio— pasa por este mismo canal.

II-D. Gestión de servicios con systemd

systemd es el gestor de inicio y de servicios de Debian: arranca los componentes del sistema de forma concurrente y describe cada proceso demonio como una *unidad* definida en un archivo de configuración [6]. Para una aplicación que se sirve por red, la unidad interesante es la de tipo `.service`, estructurada en las secciones `[Unit]` (descripción, dependencias y condiciones de arranque), `[Service]` (tipo de proceso, `ExecStart`, `ExecStop`, usuario bajo el que se ejecuta y permisos) e `[Install]` (enlaces que hacen que la unidad se active con el sistema).

El ciclo de vida de una unidad se controla con los verbos `start`, `stop`, `restart` y `status`, y `enable` y `disable` deciden si la unidad arranca con el equipo. Tras editar el archivo de una unidad es obligatorio ejecutar `systemctl daemon-reload` para que systemd lo vuelva a leer. El resultado de `status` y los mensajes que escribe la aplicación quedan en el diario del sistema, consultables con `journalctl` [6], y son la fuente de diagnóstico que la consola devuelve junto con el resultado de cada operación. Éste es exactamente el vocabulario que expone el módulo de gestión de demonios, que además transfiere a la máquina los archivos de la aplicación y ajusta sus permisos para que la unidad pueda ejecutarla.

II-E. Balanceo de carga con HAProxy

El balanceo de carga reparte las peticiones que llegan a un único punto de entrada entre varios servidores idénticos —los *back-end*— para que el servicio no dependa de una sola máquina y pueda atender más tráfico del que soporta una instancia aislada [7]. El punto de entrada que escucha en nombre de todos es el *front-end*; entre este y los servidores se decide cómo se reparten las conexiones, con algoritmos como `roundrobin`, que las distribuye de forma cíclica, `leastconn`, que las envía al servidor con menos conexiones abiertas, o `source`, que usa la dirección de origen de la petición.

HAProxy es un proxy de alto rendimiento que trabaja sobre TCP y sobre HTTP y se configura en un único archivo, `/etc/haproxy/haproxy.cfg`, dividido en las secciones `global`, `defaults`, `frontend` y `backend` [7]. Dentro de un bloque `backend`, cada servidor del balanceador se declara con una línea de la forma `server <nombre><dirección>:<puerto>`; agregar o eliminar un servidor del back-end se reduce, por tanto, a añadir o borrar esa línea y recargar la configuración. Es la operación que automatiza el cuarto módulo de la consola sobre la máquina balanceadora descrita en la precondición (b).

III. Diseño de la Solución

III-A. Escenario de Aplicación

III-A1. Descripción del Escenario: La solución se desarrolla y se prueba sobre el escenario de la Figura 1. El *equipo host* es el equipo físico donde se instala Oracle

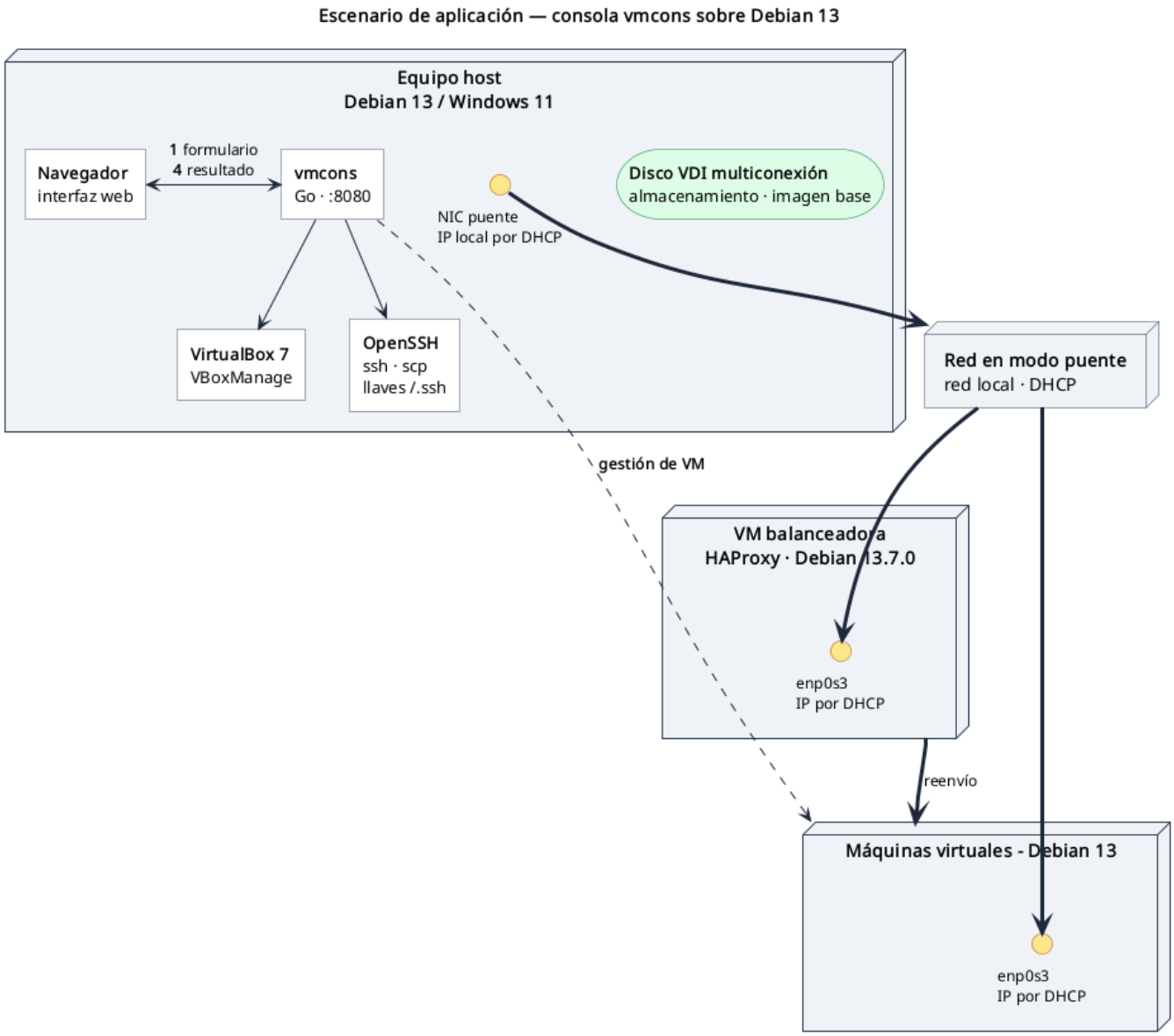


Figura 1. Escenario de pruebas: equipo host con la consola y la virtualización, máquinas virtuales Debian 13 con sus interfaces de red y direcciones por DHCP, y disco de almacenamiento; el enlace discontinuo indica la gestión de las VM por parte de la consola.

VM VirtualBox 7 y se ejecuta la consola `vmcons`; su sistema operativo puede ser GNU/Linux Debian 13 o Windows 11, ya que la consola se compila como un binario único sin dependencias en tiempo de ejecución. La consola publica la interfaz web en el puerto `8080` y es accedida desde un navegador instalado en el mismo equipo.

La red física funciona en modo puente (bridge) sobre una subred local con servidor DHCP/DNS. Ni el anfitrión ni las máquinas virtuales tienen una dirección fija: la tarjeta de red del anfitrión y la interfaz `enp0s3` de cada VM reciben la suya en cada arranque, por eso el diagrama indica «IP por DHCP» en lugar de valores concretos. Es la consola quien descubre en tiempo de ejecución la dirección de la VM con la que necesita trabajar, como se explica en la Sección III-A2. Al estar en modo puente, las VM quedan visibles en la red local con una dirección propia, lo que permite probar servicios de red reales como SSH o HAProxy sin reglas de reenvío de puertos.

Todas las máquinas virtuales ejecutan GNU/Linux Debian 13.7.0. El balanceador ocupa un nodo propio en la Figura 1 porque es la máquina del módulo de HAProxy: recibe las peticiones en su interfaz `enp0s3` y las reenvía, a través de la misma red, a los servidores del back-end; la consola se conecta también con esta misma VM para agregar o eliminar dichos servidores, como describe la Sección IV-D. Las restantes se resumen en un único contenedor del diagrama, sin fijar su número: la VM 1 es la máquina de pruebas sobre la que se ejecutan los módulos de preparación del sistema, gestión de usuarios

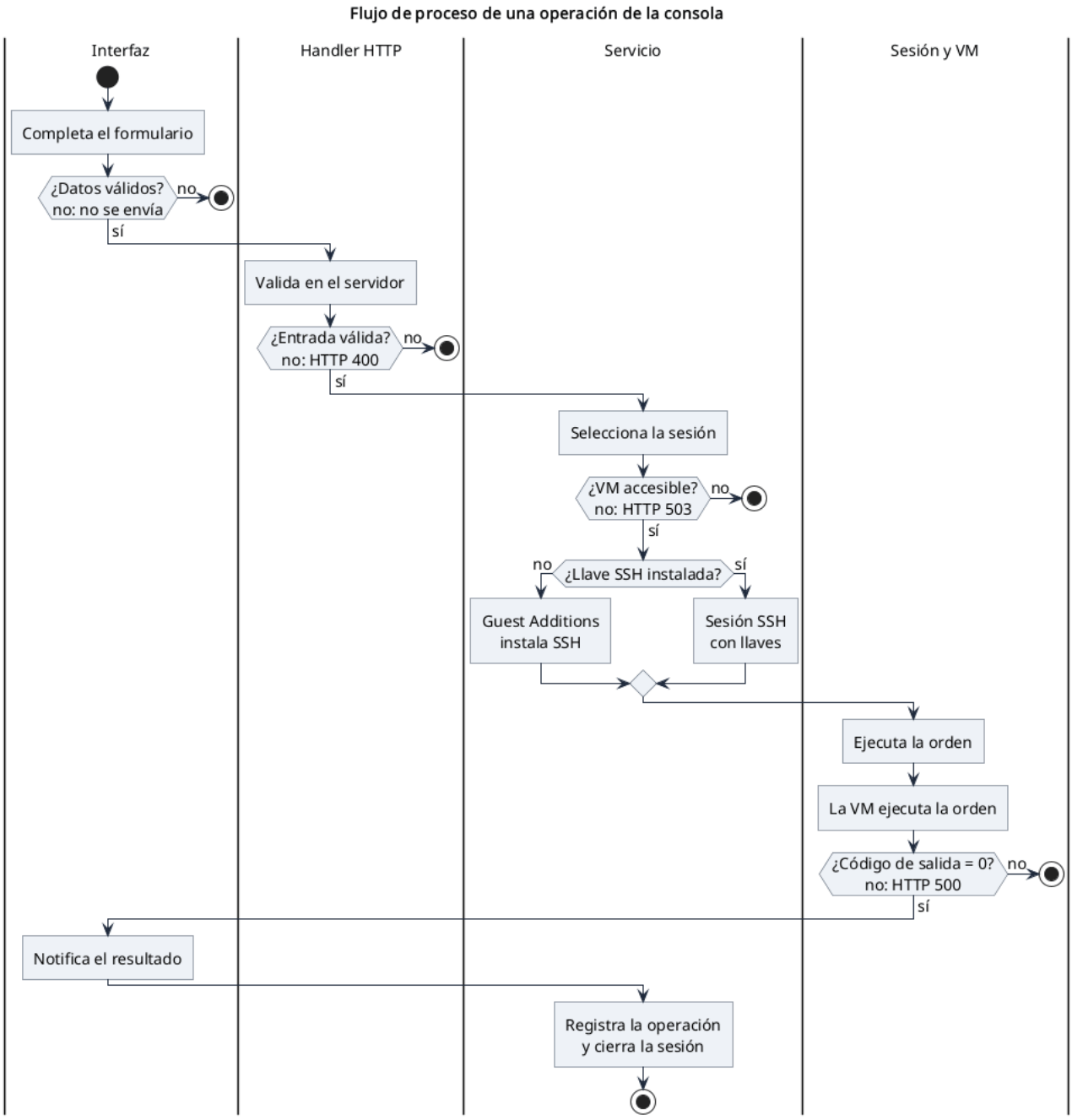


Figura 2. Flujo de proceso de una operación solicitada desde la interfaz, con el responsable de cada etapa.

y servicios con systemd, y la VM 3 y la VM 4 son los servidores del back-end; la consola abre sesión con la máquina que corresponda en cada operación, de modo que puede tratarse de cualesquiera otras. El *disco virtual VDI de multiconexión* (multiattach) que aparece dentro del equipo host es el *almacenamiento* del escenario: guarda la imagen base sobre la que se preparan, con VirtualBox y *fuera de la consola*, las back-end que se usan en las pruebas de balanceo. Como cada disco derivado carga sobre la misma imagen base, basta con duplicar el archivo para añadir otra máquina. La consola no crea ni registra máquinas virtuales: solo administra las que ya existen.

III-A2. Mecanismos de Comunicación: La consola se comunica con la máquina virtual mediante dos mecanismos distintos, recorridos por el enlace discontinuo de gestión de la Figura 1.

El **primer mecanismo** se basa en las Guest Additions [2] y se usa *únicamente antes* de instalar SSH. La consola obtiene la dirección IP de la VM con `VBoxManage guestproperty get` sobre la propiedad `/VirtualBox/GuestInfo/Net/0/V4/IP`, genera el par de llaves en el anfitrión, instala

`openssh-server`, copia la llave pública al archivo `authorized_keys` del usuario `root` y deshabilita el acceso por contraseña. Las órdenes se ejecutan con `VBoxManage guestcontrol run` y los archivos se transfieren con `guestcontrol copyto`.

El **segundo mecanismo** es el acceso por SSH con llaves de autenticación [5], mediante el cliente OpenSSH del anfitrión (`ssh` y `scp`). Una vez instalado SSH, *toda* la comunicación posterior entre la consola y la máquina se realiza por este canal, tal como exige el enunciado: el primer mecanismo deja de utilizarse y la consola trabaja de forma exclusiva sobre SSH, con la llave privada resguardada en el equipo anfitrión.

III-A3. Flujo General de la Automatización: Todo módulo de la consola recorre el mismo flujo: (1) el usuario completa el formulario en el navegador y lo envía; (2) la consola valida los datos y selecciona la sesión adecuada, es decir, decide si aún necesita Guest Additions o si ya puede trabajar por SSH; (3) ejecuta la orden de forma no interactiva sobre la máquina virtual; (4) la máquina devuelve la salida y la consola notifica el resultado, o bien presenta un mensaje de error comprensible junto con la información técnica de diagnóstico. El bucle $1 \rightarrow 4$ de la Figura 1 resume ese camino: el navegador envía el formulario y recibe el resultado, mientras que la validación, la selección de la sesión y la ejecución ocurren entre la consola y la máquina virtual.

III-B. Arquitectura de la Solución

III-B1. Visión General de la Aplicación: La consola es una aplicación web escrita en Go que se sirve con el paquete estándar `net/http`. La interfaz está organizada por módulos —preparación del sistema, gestión de usuarios, servicios con systemd y balanceo de carga—, y cada módulo presenta un formulario con los campos y las casillas de verificación que la operación requiere.

Toda operación sigue el ciclo de la Figura 2: validación en el cliente para dar respuesta inmediata, validación en el servidor sobre los parámetros crudos (que es la que realmente decide), selección de la sesión, ejecución no interactiva de la orden y notificación del resultado. Si la validación falla, la operación *no* llega a la máquina virtual y la respuesta indica el campo y el motivo; si la orden se ejecuta y termina con un código de salida distinto de cero, la respuesta conserva la orden ejecutada, el código de salida y la salida de error estándar para que el usuario pueda diagnosticar el problema. Las respuestas son documentos JSON con un campo `ok`, un `mensaje` comprensible y, cuando falla, los campos `operacion`, `codigo`, `salida`, `stderr`, `tecnico` y `campo` con el detalle técnico y el campo de validación rechazado.

III-B2. Estructura del Proyecto: El proyecto se organiza en capas siguiendo el patrón de puertos y adaptadores [8], de modo que el núcleo de la aplicación no dependa de VirtualBox, de las Guest Additions ni de OpenSSH:

Cuadro I
RECURSOS Y RUTAS DE LA API REST.

Recurso	Ruta y métodos
Sesión	`/api/session` (GET)
Paquetes	`/api/packages` (GET, POST)
Paquete	`/api/packages/{name}` (DELETE)
Llave de root	`/api/keys/root` (GET, POST)
`/etc/issue`	`/api/issue` (GET, PUT)
Hostname	`/api/hostname` (GET, PUT)
Usuarios	`/api/users` (GET, POST)
Usuario	`/api/users/{name}` (GET, DELETE)
Llave de usuario	`/api/users/{name}/key` (GET)
Unidades	`/api/units` (GET)
Unidad	`/api/units/{unit}` (GET, PUT, PATCH)
Reinicio	`/api/units/{unit}/restarts` (POST)
Servidores	`/api/haproxy/backend/servers` (GET, POST)
Servidor	`/api/haproxy/backend/servers/{server}` (DELETE)

- `main.go`: único punto de composición. Aquí se instancian los adaptadores y se inyectan a los servicios y a los manejadores HTTP.
- `internal/core/domain`: tipos del dominio (`Command`, `Result`, `Target`, `MachineState` y `ExitError`) sin dependencias externas.
- `internal/core/ports`: interfaces `Executor`, `Machine`, `Connector` y `Session` que expresan lo que la aplicación necesita del exterior.
- `internal/adapters`: las implementaciones concretas `localproc` (procesos del sistema operativo anfitrión), `vbox` (VirtualBox y Guest Additions) y `ssh` (cliente OpenSSH).
- `internal/services`: los casos de uso de cada módulo; dependen solo de los puertos del núcleo.
- `internal/handlers`: enrutan las peticiones HTTP y validan las entradas; no contienen lógica de negocio. Cada recurso de la API tiene su manejador.
- `web`: la interfaz de usuario construida con Vite y Tailwind CSS, que consume la API mediante `fetch`.
- `docs/openapi.yaml`: el contrato OpenAPI de la API, que sirve de referencia al frontend y a las pruebas de los manejadores.

Los adaptadores traducen su vocabulario al dominio: por ejemplo, `vbox` convierte el estado `poweroff` de VirtualBox en `domain.MachinePoweredOff`, de modo que el núcleo nunca necesita saber qué tecnología se está utilizando. La regla de dependencia es simple: el núcleo no importa adaptadores, los adaptadores importan el núcleo y la composición ocurre únicamente en `main.go`.

III-B3. Contrato REST de la API: La comunicación entre la interfaz y la consola sigue un estilo REST: la API expone recursos con nombre en inglés, los identificadores viajan en la ruta y las operaciones se expresan con los métodos HTTP estándar (`GET`, `POST`, `PUT`, `PATCH` y `DELETE`) en lugar de acciones dentro del cuerpo. La table I resume los recursos.

El contrato completo, con esquemas, ejemplos y códigos de respuesta, se mantiene en `docs/openapi.yaml` (OpenAPI 3.1) y se valida

con Redocly; es la fuente de verdad para el frontend y para las pruebas de los manejadores. Los códigos de estado distinguen la creación (`201` con `Location`), el reemplazo y la modificación (`200`), el borrado (`204`), la entrada inválida (`400`), el recurso inexistente (`404`), la operación aún no conectada (`501`) y la ausencia de sesión con la máquina (`503`).

Las acciones de systemd se modelan como estado del recurso: activar o detener una unidad es un `PATCH` con `activo`, habilitar o deshabilitar su arranque es un `PATCH` con `habilitado`, y el reinicio se expone como el subrecurso `restarts` porque no es idempotente. El servidor del back-end de HAProxy se identifica como `direccion:puerto`. Los cuerpos JSON conservan los nombres de campo que usa la interfaz (`nombre`, `archivos`, `activo`, `habilitado`), de modo que el formulario y el contrato no dupliquen vocabulario.

III-B4. Decisiones Tomadas Durante el Desarrollo: Las decisiones principales del diseño son las siguientes.

Binarios del sistema, no librerías nativas. El SDK de VirtualBox exige COM en Windows y bindings específicos en GNU/Linux; en su lugar se invoca `VBoxManage`, oficial y multiplataforma, y para SSH se usa el cliente incluido en el sistema en lugar de una implementación en Go [9].

Errores tipados. Las órdenes que terminan con un código distinto de cero producen un `domain.ExitError` que conserva el código, la salida estándar y la salida de error; el núcleo lo entiende sin importar qué adaptador lo originó.

Validación antes de ejecutar y en el servidor. Las direcciones se modelan con `net/netip.Addr`, los nombres de usuario y de host se comprueban con expresiones regulares y ninguna entrada cruda llega a la máquina virtual.

Un solo flujo para los siete módulos. Todos comparten el ciclo entrada, validación, sesión, ejecución, clasificación y notificación, lo que uniformiza los mensajes de error y hace que las pruebas sean repetibles.

Calidad de código verificable. El flujo de trabajo exige formateo con `gofumpt`, análisis con `golangci-lint` y pruebas con detector de condiciones de carrera antes de cada commit.

IV. Implementación

La capa HTTP está implementada: los catorce recursos de la table I tienen su manejador, la validación del servidor aplica las reglas descritas en la Sección III-B3 y cada petición se responde con el modelo JSON de la Sección III-B1. El descubrimiento de sesión expone `GET /api/session`, y los módulos de paquetes, llaves SSH y `/etc/issue` ya ejecutan operaciones reales sobre la máquina. Los demás casos de uso, mientras no se conecten, responden `501` con un mensaje comprensible, de modo que la interfaz puede integrarse sin esperar a la ejecución real. La interfaz web ya consume el contrato completo desde las ocho vistas (Sección IV-E). Las pruebas cubren el 98.4 % de los manejadores y el 100 % de los servicios.

IV-A. Preparación del Sistema

IV-A1. Instalación y Eliminación de Aplicaciones: El módulo de paquetes se apoya en el recurso `/api/packages`. El formulario indica la acción y el nombre del paquete; el servidor valida el nombre con la regla de paquetes APT antes de tocar la máquina. La instalación ejecuta `apt-get install -y` y la eliminación `apt-get remove -y` sobre ese nombre, con `DEBIAN_FRONTEND=noninteractive` para que APT no solicite confirmaciones aunque la sesión no sea interactiva.

Cada operación abre una sesión con el conector disponible, la cierra al terminar y clasifica el resultado. Si el código de salida es cero, responde `201` al instalar (con `Location`) o `204` al eliminar, e incluye la salida relevante; si es distinto de cero, responde `500` conservando la orden ejecutada, el código de salida y el `stderr` para el diagnóstico. El listado usa `dpkg-query` y filtra por el estado `install ok installed`. La cobertura del servicio de paquetes es del 100 %.

IV-A2. Configuración de la Llave de Acceso SSH para el Usuario Root: El recurso `/api/keys/root` concentra la configuración del acceso por llaves. El servicio comprueba primero que la llave privada no exista en el anfitrión y, si ya está, responde `409` para no sobrescribirla. Después abre la sesión *antes* de generar la llave: como todavía no hay llave, el descubrimiento elige Guest Additions, que es el único canal disponible.

La generación se delega en `ssh-keygen -t ed25519 -N` sobre el anfitrión. Con la llave pública ya disponible, una sola orden POSIX crea `/root/.ssh` con permisos `700`, agrega la clave a `authorized_keys` con `600`, escribe `/etc/ssh/sshd_config.d/99-vmcons.conf` con `PasswordAuthentication no` y `PermitRootLogin prohibit-password`, y recarga el servicio con `systemctl reload ssh`. Al cerrar la sesión, el conector `ssh` queda instalado y pasa a ser el preferido para las siguientes operaciones.

La llave privada se entrega con `GET /api/keys/root` como archivo adjunto (`Content-Disposition: attachment`) y nunca se escribe en los logs ni en el cuerpo JSON; si todavía no existe, la respuesta es `404`. La interfaz muestra un aviso para resguardarla y no compartirla.

IV-A3. Personalización de la Entrada del Sistema: El recurso `/api/issue` lee y reemplaza el banner que `agetty` muestra antes del inicio de sesión. El formulario envía el mensaje personalizado y las casillas de nombre de host, fecha, hora y dirección IP; el servidor exige que haya al menos un elemento y rechaza mensajes de más de 512 caracteres, con NUL o con saltos de línea, porque el formato es de una línea por elemento.

El servicio escribe el archivo con las secuencias de `agetty` para nombre de host, fecha, hora y dirección

IP, precedidas por una etiqueta legible. Al leer, reconoce esas etiquetas y reconstruye las casillas; si el archivo no sigue el formato de la consola —por ejemplo, el `/etc/issue` original de Debian— todo el contenido se devuelve como mensaje y las opciones quedan desactivadas, de modo que no se pierde un banner ajeno. La escritura usa una sola orden POSIX con `printf` y deja permisos `644`.

IV-A4. Asignación del Nombre de Host y Actualización del Archivo de Hosts: El recurso `/api/hostname` consulta y reemplaza el nombre de la máquina y su entrada en `/etc/hosts`. Al leer, el servicio toma el nombre de `/etc/hostname`, la primera IPv4 que devuelve `hostname -I` y comprueba si el nombre aparece como destino en `/etc/hosts`; si no hay una IPv4 disponible, responde con un error comprensible en lugar de inventar una dirección.

Al escribir, el servidor valida el nombre como RFC 1123 y la IP con `netip.Addr` antes de ejecutar nada. Después ejecuta `hostnamectl set-hostname` y, si la casilla está marcada, reescribe `/etc/hosts`: elimina las líneas que mapean el nombre, conserva el resto de entradas (incluido `localhost`) y agrega «ip nombre» al final. El cambio y la actualización del archivo se ejecutan en una sola orden POSIX, de modo que un fallo intermedio no deja el sistema a medio configurar.

IV-B. Gestión de Usuarios

IV-B1. Creación de Usuarios y Privilegios de sudo:

IV-B2. Eliminación de Usuarios:

IV-C. Gestión de Demonios con systemd

IV-D. Gestión de un Balanceador de Carga con HAProxy

IV-E. Interfaz Web

La interfaz es una aplicación de una sola página construida con Vite, JavaScript sin *framework* y Tailwind CSS. Se prescindió de una biblioteca de componentes porque la interfaz es un conjunto fijo de formularios y tablas: un enrutador por fragmento de URL y funciones que construyen el árbol del documento bastan para el alcance del enunciado y evitan incorporar un *runtime* adicional.

Armazón. Una sola página con nueve pestañas —los ocho módulos del enunciado y el historial—, una barra superior con el nombre de la máquina activa, su dirección y el estado de la sesión, y un panel lateral con el resultado de la última operación: mensaje, orden ejecutada, código de salida, salida estándar, salida de error y diagnóstico. Las pestañas no recargan el documento: cada vista consulta su recurso del Sección III-B3 al montarse. El historial se muestra deshabilitado porque su persistencia está diferida, tal como se registró en `docs/TODO.md`.

Sistema de diseño. La apariencia adopta la ergonomía de una herramienta de escritorio en lugar de la de un sitio web: lienzo gris, blanco reservado para las superficies operativas, bordes de un píxel, sin sombras ni degradados, controles de 26 píxeles y un rojo (`#C0392B`) que marca la pestaña activa, el foco y las acciones destructivas. Los valores —color, tipografía, radios y espaciado— quedaron en `docs/stitch/DESIGN.md` como tokens, de modo que la vista previa del diseño y la implementación comparten la misma paleta. Los iconos son Lucide de 16 píxeles con trazo de 1.5.

Cliente de la API. Un único módulo (`web/src/api.js`) concentra las rutas, los cuerpos y los errores del contrato; ninguna vista invoca `fetch` directamente. Los fallos se normalizan en un tipo `ApiError` que conserva el código HTTP, el mensaje comprensible, el campo rechazado y el detalle técnico. Una respuesta `501` se presenta como módulo aún no conectado, no como error de red, lo que permitió integrar las ocho vistas mientras los casos de uso del servidor se completaban uno por uno. Las llaves privadas se descargan como *Blob* y se entregan con el nombre de archivo sugerido, y las opciones de una operación viajan en la cadena de consulta, por ejemplo `?files=true`.

Estados de las vistas. Toda vista con datos remotos sigue el mismo ciclo: consulta al montarse, fila de espera, mensaje de lista vacía, mensaje de error con la causa y, por último, los datos. Las operaciones de escritura envían su resultado al panel lateral y, si terminan bien, vuelven a consultar el recurso afectado, de modo que la tabla nunca queda desactualizada respecto de la máquina.

Validación en el cliente. Las expresiones del contrato —nombre de paquete, nombre de usuario, nombre de host, unidad `.service`, IPv4, puerto y modo octal— se replican en `web/src/validacion.js` para dar respuesta inmediata; la validación del servidor sigue siendo la que decide.

Empaquetado. Vite compila la interfaz en `web/dist`, que se versiona y se embebe en el binario con `//go:embed`: la consola sirve la aplicación en la raíz y los recursos con huella en `/assets/`, en el mismo origen que la API. Las tipografías y los iconos viajan dentro del binario, de modo que la interfaz se ve igual en un equipo sin salida a Internet. La tarea `build` del `Taskfile` recompila el cliente antes de enlazar el binario de Go, para que el ejecutable nunca quede con una versión anterior de la vista.

Pruebas. El cliente se verifica con Vitest: cuarenta y dos casos sobre la construcción de las peticiones, la traducción de errores, las reglas de validación, las utilidades de presentación y el montaje de las vistas en un DOM simulado. El empaquetado importa solo los iconos utilizados, con lo que el JavaScript entregado al navegador es de 64 kB en lugar de los 528 kB que resultaban de importar el catálogo completo.

V. Pruebas

V-A. Diseño de las Pruebas

V-B. Resultados y Evidencias

VI. Conclusiones

VI-A. Problemas Encontrados y Soluciones

VI-B. Herramientas de Inteligencia Artificial Utilizadas

VI-C. Conclusiones del Trabajo

Referencias

[1] *Oracle VM VirtualBox User Manual*, 7.1, Oracle, 2025. dirección: https://www.virtualbox.org/manual/

[2] *VirtualBox Guest Additions*, Oracle, 2025. dirección: https://www.virtualbox.org/manual/ch04.html

[3] *Debian Reference*, 13 (trixie), Debian Project, 2025. dirección: https://www.debian.org/doc/manuals/debian-reference/

[4] *APT — Advanced Package Tool*, Debian Project, 2025. dirección: https://manpages.debian.org/apt

[5] T. Ylonen y C. Lonvick, *The Secure Shell (SSH) Authentication Protocol*, RFC 4252, Internet Engineering Task Force, 2006. dirección: https://www.rfc-editor.org/rfc/rfc4252

[6] *systemd.service — Service unit configuration*, freedesktop.org, 2025. dirección: https://www.freedesktop.org/software/systemd/man/systemd.service.html

[7] *HAProxy Documentation*, HAProxy Technologies, 2025. dirección: https://docs.haproxy.org/

[8] A. Cockburn, *Hexagonal Architecture*, 2005. dirección: https://alistair.cockburn.us/hexagonal-architecture

[9] A. A. A. Donovan y B. W. Kernighan, *The Go Programming Language*. Boston: Addison-Wesley, 2015.